```
from google.colab import drive
drive.mount('/content/drive')
```

```
Mounted at /content/drive
```

```
import numpy as np
import os

base_path = "/content/drive/MyDrive/PROJECT PHASE2/Embeddings"

# Original training data
X_original = np.load(os.path.join(base_path, "X_train_embeddings.npy"))
y_original = np.load(os.path.join(base_path, "y_train.npy"))

# SMOTE
X_smote = np.load(os.path.join(base_path, "X_train_smote.npy"))
y_smote = np.load(os.path.join(base_path, "y_train_smote.npy"))

# ADASYN
X_adasyn = np.load(os.path.join(base_path, "X_train_adasyn.npy"))
y_adasyn = np.load(os.path.join(base_path, "y_train_adasyn.npy"))

# SMOTE-ENN
X_smote_enn = np.load(os.path.join(base_path, "X_train_smote_enn.npy"))
y_smote_enn = np.load(os.path.join(base_path, "y_train_smote_enn.npy"))

# SMOTE-Tomek
X_smote_tomek = np.load(os.path.join(base_path, "X_train_smote_tomek.npy"))
y_smote_tomek = np.load(os.path.join(base_path, "y_train_smote_tomek.npy"))

print("Original    :", X_original.shape, y_original.shape)
print("SMOTE       :", X_smote.shape, y_smote.shape)
print("ADASYN      :", X_adasyn.shape, y_adasyn.shape)
print("SMOTE-ENN   :", X_smote_enn.shape, y_smote_enn.shape)
print("SMOTE-Tomek :", X_smote_tomek.shape, y_smote_tomek.shape)
```

```
Original    : (195072, 768) (195072,)
SMOTE       : (202711, 768) (202711,)
ADASYN      : (202623, 768) (202623,)
SMOTE-ENN   : (192315, 768) (192315,)
SMOTE-Tomek : (201625, 768) (201625,)
```

```
from collections import Counter

# Count samples in each class
class_counts = Counter(y_original)

# Sort classes from rarest to most frequent
sorted_counts = sorted(class_counts.items(), key=lambda x: x[1])

print("Rarest classes:")
for class_id, count in sorted_counts[:15]:
    print(f"Class {class_id}: {count} samples")
```

```
Rarest classes:
Class 31: 1 samples
Class 8: 4 samples
Class 20: 4 samples
Class 1: 14 samples
Class 13: 14 samples
Class 28: 17 samples
Class 24: 21 samples
Class 44: 21 samples
Class 27: 35 samples
Class 29: 51 samples
Class 37: 57 samples
Class 33: 71 samples
Class 34: 75 samples
Class 0: 120 samples
Class 12: 120 samples
```

```
# Compare selected minority-class counts before and after resampling

selected_classes = [37, 29, 33, 34, 0]  # S-BIOP, I-MACEQ, I-PARA, I-PRO, B-APPL

datasets = {
    "Original": y_original,
    "SMOTE": y_smote,
    "ADASYN": y_adasyn,
    "SMOTE-ENN": y_smote_enn,
    "SMOTE-Tomek": y_smote_tomek
}
```

```
for class_id in selected_classes:
    print(f"\nClass {class_id}:")

    for name, labels in datasets.items():
        count = np.sum(labels == class_id)
        print(f"  {name:<12}: {count}")
```

```
Class 37:
  Original    : 57
  SMOTE       : 500
  ADASYN      : 505
  SMOTE-ENN   : 498
  SMOTE-Tomek : 500

Class 29:
  Original    : 51
  SMOTE       : 500
  ADASYN      : 496
  SMOTE-ENN   : 495
  SMOTE-Tomek : 500

Class 33:
  Original    : 71
  SMOTE       : 500
  ADASYN      : 500
  SMOTE-ENN   : 498
  SMOTE-Tomek : 500

Class 34:
  Original    : 75
  SMOTE       : 500
  ADASYN      : 498
  SMOTE-ENN   : 489
  SMOTE-Tomek : 500

Class 0:
  Original    : 120
  SMOTE       : 500
  ADASYN      : 496
  SMOTE-ENN   : 485
  SMOTE-Tomek : 498
```

```
from sklearn.decomposition import PCA
import numpy as np

# ------------------------------------------------------------
# Analyse one minority class first: Class 37 = S-BIOP
# ------------------------------------------------------------

class_id = 37

# Keep only the selected class from each dataset
original_mask = (y_original == class_id)
smote_mask = (y_smote == class_id)
adasyn_mask = (y_adasyn == class_id)
smote_enn_mask = (y_smote_enn == class_id)
smote_tomek_mask = (y_smote_tomek == class_id)

X_orig_class = X_original[original_mask]
X_smote_class = X_smote[smote_mask]
X_adasyn_class = X_adasyn[adasyn_mask]
X_smote_enn_class = X_smote_enn[smote_enn_mask]
X_smote_tomek_class = X_smote_tomek[smote_tomek_mask]

print("S-BIOP samples:")
print("Original     :", len(X_orig_class))
print("SMOTE        :", len(X_smote_class))
print("ADASYN       :", len(X_adasyn_class))
print("SMOTE-ENN    :", len(X_smote_enn_class))
print("SMOTE-Tomek  :", len(X_smote_tomek_class))

# Combine all S-BIOP embeddings
X_combined = np.vstack([
    X_orig_class,
    X_smote_class,
    X_adasyn_class,
    X_smote_enn_class,
    X_smote_tomek_class
])

# Fit ONE common PCA transformation
pca = PCA(n_components=2, random_state=42)
X_2d = pca.fit_transform(X_combined)

print("\nCombined shape:", X_combined.shape)
print("2D shape      :", X_2d.shape)
```

```
print("Explained variance ratio:", pca.explained_variance_ratio_)
print("Total variance explained:", pca.explained_variance_ratio_.sum())
```

```
S-BIOP samples:
Original     : 57
SMOTE        : 500
ADASYN       : 505
SMOTE-ENN    : 498
SMOTE-Tomek  : 500

Combined shape: (2060, 768)
2D shape      : (2060, 2)
Explained variance ratio: [0.35411796 0.1621377 ]
Total variance explained: 0.5162557
```

```
import matplotlib.pyplot as plt

# Get the 2D coordinates for each dataset
n_orig = len(X_orig_class)
n_smote = len(X_smote_class)
n_adasyn = len(X_adasyn_class)
n_smote_enn = len(X_smote_enn_class)

X_orig_2d = X_2d[:n_orig]

X_smote_2d = X_2d[n_orig:n_orig + n_smote]

X_adasyn_2d = X_2d[
    n_orig + n_smote:
    n_orig + n_smote + n_adasyn
]

X_smote_enn_2d = X_2d[
    n_orig + n_smote + n_adasyn:
    n_orig + n_smote + n_adasyn + n_smote_enn
]

X_smote_tomek_2d = X_2d[
    n_orig + n_smote + n_adasyn + n_smote_enn:
]

# Plot
plt.figure(figsize=(10, 8))

plt.scatter(
    X_orig_2d[:, 0],
    X_orig_2d[:, 1],
    s=80,
    label="Original S-BIOP"
)

plt.scatter(
    X_smote_2d[:, 0],
    X_smote_2d[:, 1],
    s=15,
    alpha=0.5,
    label="SMOTE"
)

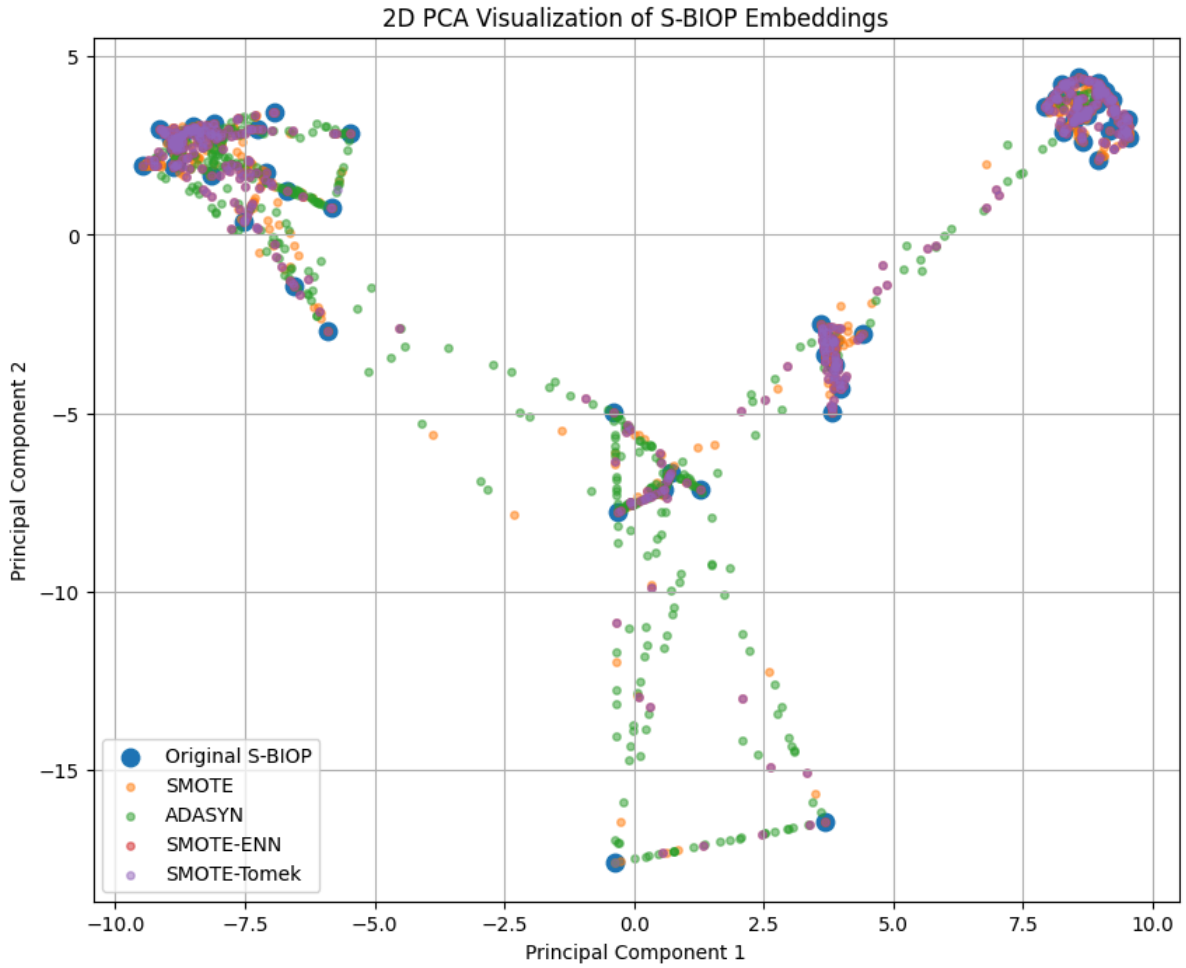
plt.scatter(
    X_adasyn_2d[:, 0],
    X_adasyn_2d[:, 1],
    s=15,
    alpha=0.5,
    label="ADASYN"
)

plt.scatter(
    X_smote_enn_2d[:, 0],
    X_smote_enn_2d[:, 1],
    s=15,
    alpha=0.5,
    label="SMOTE-ENN"
)

plt.scatter(
    X_smote_tomek_2d[:, 0],
    X_smote_tomek_2d[:, 1],
    s=15,
    alpha=0.5,
    label="SMOTE-Tomek"
)

plt.xlabel("Principal Component 1")
plt.ylabel("Principal Component 2")
```

```
plt.title("2D PCA Visualization of S-BIOP Embeddings")
plt.legend()
plt.grid(True)
plt.show()
```



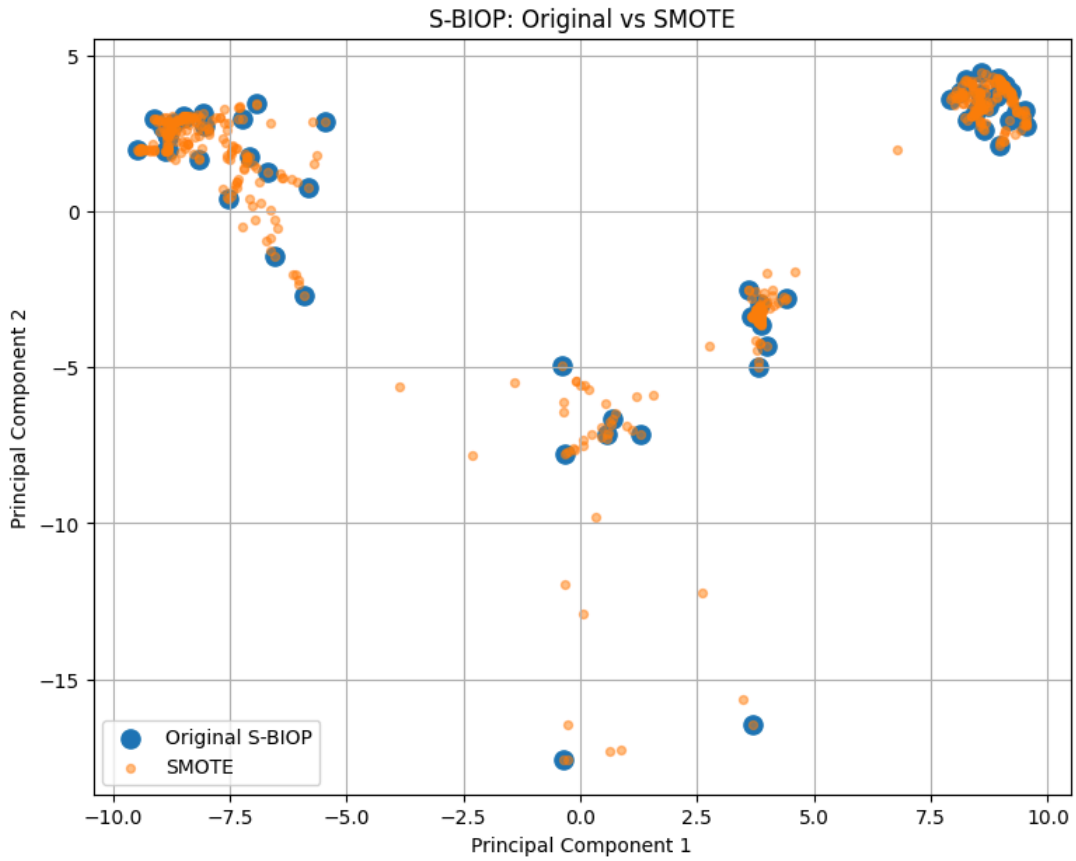

```
import matplotlib.pyplot as plt

plt.figure(figsize=(9, 7))

# Original S-BIOP points
plt.scatter(
    X_orig_2d[:, 0],
    X_orig_2d[:, 1],
    s=90,
    label="Original S-BIOP"
)

# SMOTE points
plt.scatter(
    X_smote_2d[:, 0],
    X_smote_2d[:, 1],
    s=18,
    alpha=0.5,
    label="SMOTE"
)

plt.xlabel("Principal Component 1")
plt.ylabel("Principal Component 2")
plt.title("S-BIOP: Original vs SMOTE")
plt.legend()
plt.grid(True)
plt.show()
```

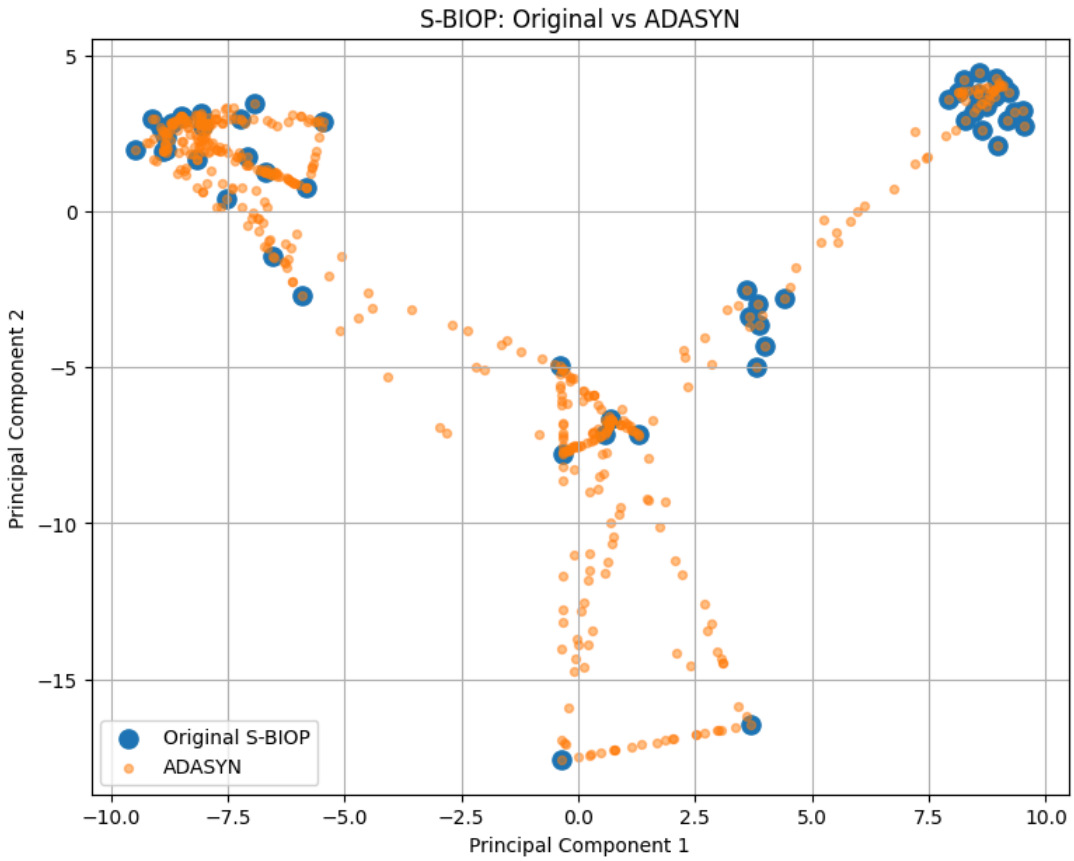



```
plt.figure(figsize=(9, 7))

# Original S-BIOP points
plt.scatter(
    X_orig_2d[:, 0],
    X_orig_2d[:, 1],
    s=90,
    label="Original S-BIOP"
)

# ADASYN points
plt.scatter(
    X_adasyn_2d[:, 0],
    X_adasyn_2d[:, 1],
    s=18,
    alpha=0.5,
    label="ADASYN"
)

plt.xlabel("Principal Component 1")
plt.ylabel("Principal Component 2")
plt.title("S-BIOP: Original vs ADASYN")
plt.legend()
plt.grid(True)
plt.show()
```

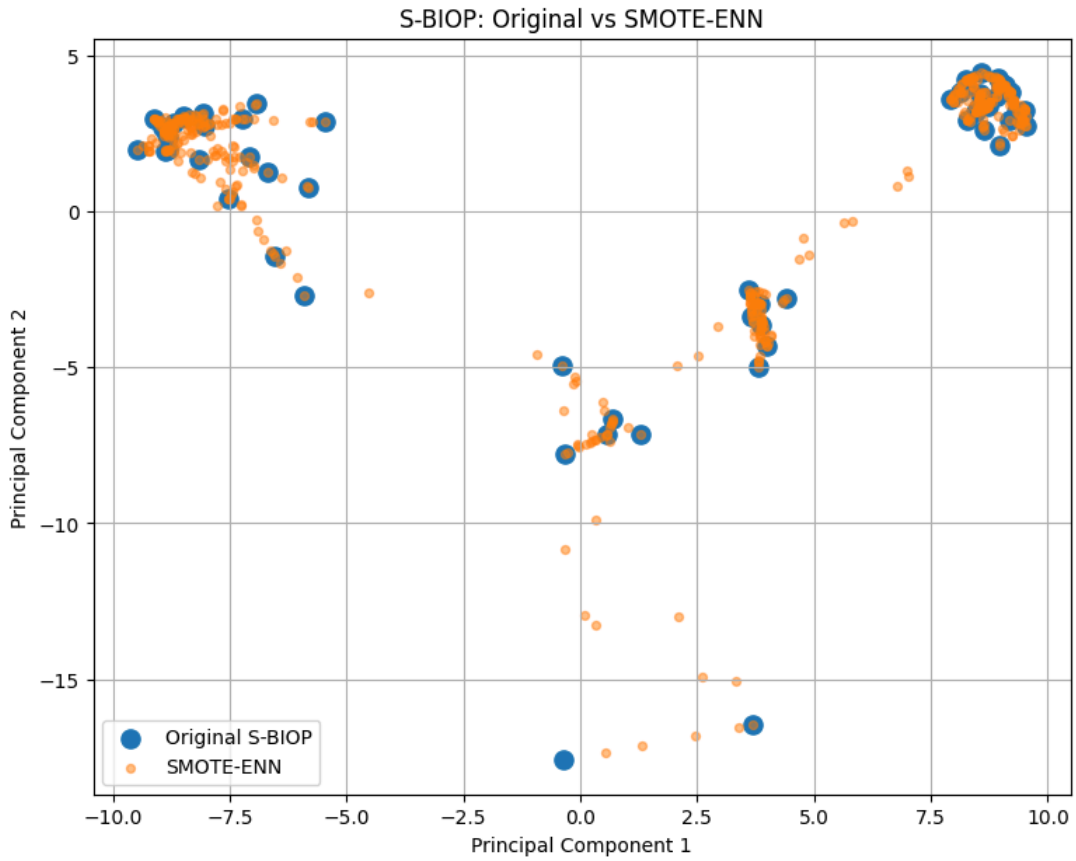



```
plt.figure(figsize=(9, 7))

# Original S-BIOP points
plt.scatter(
    X_orig_2d[:, 0],
    X_orig_2d[:, 1],
    s=90,
    label="Original S-BIOP"
)

# SMOTE-ENN points
plt.scatter(
    X_smote_enn_2d[:, 0],
    X_smote_enn_2d[:, 1],
    s=18,
    alpha=0.5,
    label="SMOTE-ENN"
)

plt.xlabel("Principal Component 1")
plt.ylabel("Principal Component 2")
plt.title("S-BIOP: Original vs SMOTE-ENN")
plt.legend()
plt.grid(True)
plt.show()
```

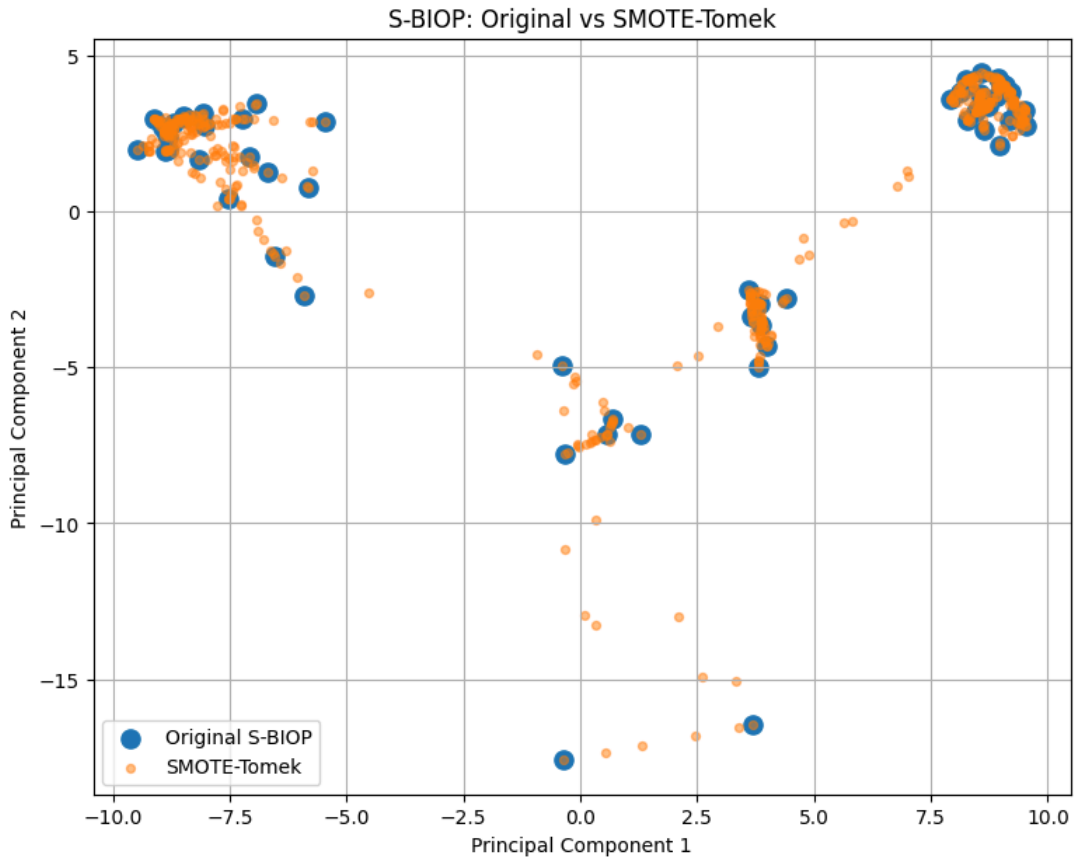



```
plt.figure(figsize=(9, 7))

# Original S-BIOP points
plt.scatter(
    X_orig_2d[:, 0],
    X_orig_2d[:, 1],
    s=90,
    label="Original S-BIOP"
)

# SMOTE-Tomek points
plt.scatter(
    X_smote_tomek_2d[:, 0],
    X_smote_tomek_2d[:, 1],
    s=18,
    alpha=0.5,
    label="SMOTE-Tomek"
)

plt.xlabel("Principal Component 1")
plt.ylabel("Principal Component 2")
plt.title("S-BIOP: Original vs SMOTE-Tomek")
plt.legend()
plt.grid(True)
plt.show()
```




```
from sklearn.neighbors import NearestNeighbors
import numpy as np

# Use the ORIGINAL S-BIOP embeddings as the reference
reference = X_orig_class

# Fit nearest-neighbour model in the original 768-D space
nn = NearestNeighbors(n_neighbors=1, metric="euclidean")
nn.fit(reference)

# Calculate nearest distance for each resampled S-BIOP point
datasets_for_distance = {
    "SMOTE": X_smote_class,
    "ADASYN": X_adasyn_class,
    "SMOTE-ENN": X_smote_enn_class,
    "SMOTE-Tomek": X_smote_tomek_class
}

print("Nearest-neighbour distance to original S-BIOP samples")
print("-" * 55)

for name, X_data in datasets_for_distance.items():
    distances, _ = nn.kneighbors(X_data)

    print(f"{name:<15} Mean   : {distances.mean():.4f}")
    print(f"{'':<15} Median : {np.median(distances):.4f}")
    print(f"{'':<15} Max    : {distances.max():.4f}")
    print()
```

```
Nearest-neighbour distance to original S-BIOP samples
-------------------------------------------------------
SMOTE           Mean   : 1.9303
                Median : 1.5273
                Max    : 10.6429

ADASYN          Mean   : 2.9345
                Median : 2.4338
                Max    : 12.0627

SMOTE-ENN       Mean   : 2.0625
                Median : 1.6884
                Max    : 9.9368

SMOTE-Tomek     Mean   : 2.0601
                Median : 1.6884
                Max    : 9.9368
```

```
# ------------------------------------------------------------
# Nearest-neighbour distance for SYNTHETIC samples only
```

```
# ------------------------------------------------------------

def synthetic_only_distances(X_resampled, X_original_class):
    """
    Remove samples that are identical to original samples,
    then calculate distance to the nearest original minority sample.
    """

    # Find nearest original sample and its distance
    nn = NearestNeighbors(n_neighbors=1, metric="euclidean")
    nn.fit(X_original_class)

    distances, _ = nn.kneighbors(X_resampled)
    distances = distances.ravel()

    # Keep only samples that are not identical to an original sample
    synthetic_mask = distances > 1e-8
    synthetic_distances = distances[synthetic_mask]

    return synthetic_distances


synthetic_distance_results = {}

for name, X_data in datasets_for_distance.items():
    distances = synthetic_only_distances(
        X_data,
        X_orig_class
    )

    synthetic_distance_results[name] = distances

    print(f"{name:<15}")
    print(f"  Synthetic samples : {len(distances)}")
    print(f"  Mean distance     : {distances.mean():.4f}")
    print(f"  Median distance   : {np.median(distances):.4f}")
    print(f"  Maximum distance  : {distances.max():.4f}")
    print()
```

```
SMOTE
  Synthetic samples : 499
  Mean distance     : 1.9342
  Median distance   : 1.5328
  Maximum distance  : 10.6429

ADASYN
  Synthetic samples : 505
  Mean distance     : 2.9345
  Median distance   : 2.4338
  Maximum distance  : 12.0627

SMOTE-ENN
  Synthetic samples : 497
  Mean distance     : 2.0666
  Median distance   : 1.6903
  Maximum distance  : 9.9368

SMOTE-Tomek
  Synthetic samples : 499
  Mean distance     : 2.0643
  Median distance   : 1.6903
  Maximum distance  : 9.9368
```

```
# Inspect the smallest nearest-neighbour distances
# for the SMOTE S-BIOP samples

nn = NearestNeighbors(n_neighbors=1, metric="euclidean")
nn.fit(X_orig_class)

smote_distances, _ = nn.kneighbors(X_smote_class)
smote_distances = smote_distances.ravel()

sorted_distances = np.sort(smote_distances)

print("Smallest 20 nearest-neighbour distances for SMOTE:")
print(sorted_distances[:20])

print("\nDistance statistics:")
print("Minimum :", sorted_distances.min())
print("Maximum :", sorted_distances.max())
```

```
Smallest 20 nearest-neighbour distances for SMOTE:
[0.00000000e+00 3.37174782e-07 3.37174782e-07 3.37174782e-07
 3.37174782e-07 3.37174782e-07 3.37174782e-07 3.37174782e-07
 4.76837158e-07 4.76837158e-07 4.76837158e-07 4.76837158e-07
```

```
 4.76837158e-07 4.76837158e-07 4.76837158e-07 4.76837158e-07
 4.76837158e-07 4.76837158e-07 4.76837158e-07 4.76837158e-07]

Distance statistics:
Minimum : 0.0
Maximum : 10.64294147491455
```

```
# ------------------------------------------------------------
# Corrected nearest-neighbour analysis
# Synthetic samples are identified using a numerical tolerance
# ------------------------------------------------------------

TOLERANCE = 1e-5

def get_synthetic_distances(X_resampled, X_original_class):
    """
```